

Лабораторная работа №1

Литературное программирование

Бабенко Артём Сергеевич

Содержание

1	Цель работы	4
2	Задание	5
3	Теоретическое введение	6
4	Выполнение лабораторной работы	7
5	Экспоненциальный рост	13
5.1	Инициализация проекта и загрузка пакетов	13
5.2	Определение модели	13
5.3	Первый запуск с параметрами по умолчанию	14
5.4	Визуализация результатов	16
5.5	Анализ результатов	17
5.6	Сохранение всех результатов	17
6	Параметрическое исследование экспоненциального роста	19
6.1	Активация проекта и загрузка пакетов	19
6.2	Определение модели	20
6.3	Определение параметров в Dict	20
6.4	Функция-обертка для запуска одного эксперимента	21
6.5	Запуск базового эксперимента	22
6.6	Визуализация базового эксперимента	23
6.7	Параметрическое сканирование	24
6.8	Запуск всех экспериментов и сбор результатов	26
6.9	Анализ и визуализация результатов сканирования	27
6.10	Бенчмаркинг с разными параметрами	31
6.11	Сохранение всех результатов	35
7	Выводы	37
	Список литературы	38

Список иллюстраций

4.1	«Установка DrWatson»	7
4.2	«Добавление необходимых пакетов»	7
4.3	«Добавление необходимых пакетов»	8
4.4	«Итоговая структура проекта»	8
4.5	«Скрипт scripts/01_exponential_growth.jl»	9
4.6	«Скрипт scripts/tangle.jl»	9
4.7	Формат ipynb	10
4.8	Формат qmd	11
4.9	«Скрипт scripts/02_exponential_growth.jl»	12
4.10	«Литературная реализация модели»	12

1 Цель работы

Изучить принципы воспроизводимых научных вычислений с использованием языка программирования Julia и пакетов DrWatson и Literate, а также освоить автоматическую генерацию различных форматов отчётов (скриптов, ноутбуков и документов Quarto) из единого исходного literate-скрипта.

2 Задание

- Изучить структуру научного проекта, организованного с использованием пакета DrWatson.
- Освоить принципы literate-программирования, объединяющего программный код и документацию.
- Создать literate-скрипт, содержащий код модели и пояснения.
- Реализовать скрипт генерации производных форматов с использованием пакета Literate.

3 Теоретическое введение

В современных научных исследованиях важным требованием является воспроизводимость результатов вычислений. Это означает, что любой исследователь должен иметь возможность повторить вычисления и получить те же результаты, используя исходный код и данные.

Одним из подходов к обеспечению воспроизводимости является `literate`-программирование. Данный подход предполагает объединение программного кода и текстового описания в одном документе. Это позволяет одновременно документировать алгоритм и реализовывать его программно. В языке Julia для этого используется пакет `Literate.jl`, который позволяет автоматически преобразовывать исходный `literate`-скрипт в различные форматы, такие как обычный скрипт, Jupyter notebook и документы Quarto[2; 3].

Для организации структуры научного проекта используется пакет `DrWatson.jl`. Он обеспечивает стандартизированную структуру каталогов проекта, управление зависимостями и упрощает воспроизводимость вычислений[1].

Использование инструментов `Literate` и `DrWatson` позволяет автоматизировать процесс подготовки научных отчётов, обеспечить согласованность кода и документации, а также повысить прозрачность и воспроизводимость вычислительных экспериментов.

4 Выполнение лабораторной работы

Создадим проект DrWatson для лабораторных. Для этого сначала установим необходимый пакет и инициализируем проект(рис. 4.1).

```
Activating new project at ~/work/study/2026-1/2026-1==study--security-modeling/2026-1--study--security-modeling/labs/lab01/project
Resolving package versions...
Cloning [93885bb-8acc-5ee8-8aee-3f567f8a3819] JLD2 from https://github.com/JuliaIO/JLD2.jl.git
Installed JLD2 v0.6.3
Cloning [668a2e73-6563-6170-7388-637461726353] Scratch from https://github.com/JuliaPackaging/Scratch.jl.git
Installed Scratch v1.3.0
Cloning [21216c6a-2e73-6563-6e65-726566657250] Preferences from https://github.com/JuliaPackaging/Preferences.jl.git
Installed Preferences v1.5.1
Cloning [4c0b8ee4-ad6c-4d73-81a0-b6caacae83c8] ChunkCodeclibZlib from https://github.com/JuliaIO/ChunkCodecs.jl.git
Installed ChunkCodeclibZlib v1.0.0
Cloning [634d3b9d-ee7a-5ddf-bec9-22491ea816e1] DrWatson from https://github.com/JuliaDynamics/DrWatson.jl.git
Installed DrWatson v2.19.1
Cloning [692b3bcd-3c85-4b1f-b188-f13ce0eb3210] JLLWrappers from https://github.com/JuliaPackaging/JLLWrappers.jl.git
Installed JLLWrappers v1.7.1
Cloning [f5f93275-4c33-4d17-b943-31286c8f4001] ChunkCodeclibZlib from https://github.com/JuliaIO/ChunkCodecs.jl.git
```

Рисунок 4.1: «Установка DrWatson»

Создадим файл `add_packages.jl` в корне проекта для добавления необходимых пакетов(рис. 4.2, рис. 4.3).

```
add_packages.jl X
add_packages.jl
1  #!/usr/bin/env julia
   Run Cell | Run Above
2  ## add_packages.jl
3  using Pkg
4  Pkg.activate(".") # Активируем текущий проект
   Run Cell | Run Above
5  ## ОСНОВНЫЕ ПАКЕТЫ ДЛЯ РАБОТЫ
6  packages = [
7      "DrWatson", # Организация проекта
8      "DifferentialEquations", # Решение ОДУ
9      "Plots", # Визуализация
10     "DataFrames", # Таблицы данных
11     "CSV", # Работа с CSV
12     "JLD2", # Сохранение данных
13     "Literate", # Literate programming
14     "IJulia", # Jupyter notebook
15     "BenchmarkTools", # Бенчмаркинг
16     "Quarto" # Создание отчетов
17 ]
18 println("Установка базовых пакетов...")
19 Pkg.add(packages)
20 println("\n✅ Все пакеты установлены!")
21 println("Для проверки: using DrWatson, DifferentialEquations, Plots")
```

Рисунок 4.2: «Добавление необходимых пакетов»

```
[e8c9d7f7] + PCRE2_jll v10.42.0+1
[bea87d4e] + SuiteSparse_jll v7.7.0+0
[8e850b99] + libblastrampoline_jll v5.11.0+0
Info Packages marked with ^ and x have new versions available. Those with ^ may be upgradable, but those with x are restricted by compatibility constraints.
Building Conda → ~/.julia/scratchspaces/44cfe95a-1eb2-52ea-b672-e2afdf69b78f/8f86b0cfa4c514c7b9546756dba91fc9bc92dc9/build.log
Building IJulia → ~/.julia/scratchspaces/44cfe95a-1eb2-52ea-b672-e2afdf69b78f/4989c2fbc99972c57e0921b6848f8669abc58a34/build.log
Precompiling project...
377 dependencies successfully precompiled in 237 seconds. 121 already precompiled.
✓ Все пакеты установлены!
Для проверки использовать DevTools, DifferentialEquations, Plots
Manifest.toml
Project.toml
```

Рисунок 4.3: «Добавление необходимых пакетов»

Итоговая структура проекта выглядит следующим образом(рис. 4.4).

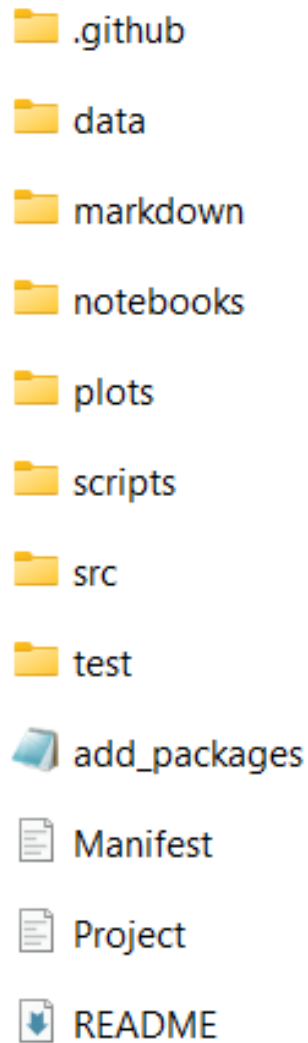


Рисунок 4.4: «Итоговая структура проекта»

В качестве примера выполнения работы используем модель экспоненциального роста. Создадим скрипт (scripts/01_exponential_growth.jl) и выполним его(рис. 4.5).


```

Admin@DESKTOP-KPFJU04 MINGW64 ~/work/study/2026-1/2026-1==study--security-modeling/labs/lab01/project (master)
$ julia --project=. scripts/01_exponential_growth.jl
Первые 5 строк результатов:
5x2 DataFrame
  Row    t      u
  Float64 Float64
1      0.0  1.0
2      0.1  1.03045
3      0.2  1.06184
4      0.3  1.09417
5      0.4  1.1275
Аналитическое время удвоения: 2.31

```

Рисунок 4.5: «Скрипт scripts/01_exponential_growth.jl»

Также создадим скрипт для генерации производных форматов (scripts/tangle.jl)рис. 4.6. В результате получим код оформленный в разных форматах(рис. 4.7, рис. 4.8)

```

1  #!/usr/bin/env julia
2  # tangle.jl - Генератор отчетов из Literate-скриптов
3  # Использование: julia tangle.jl <путь_к_скрипту>
4
5  using DrWatson
6  @quickactivate # Активирует текущий проект DrWatson
7
8  using Literate
9  function main()
10     if length(ARGS) == 0
11         println("""
12             Использование: julia tangle.jl <путь_к_скрипту>
13             Примеры:
14             julia tangle.jl scripts/lab1.jl
15             """)
16         return
17     end
18     script_path = ARGS[1]
19     if !isfile(script_path)
20         error("Файл не найден: $script_path")
21     end
22     # Пути и имена
23     script_dir = dirname(script_path)
24     script_name = splitext(basename(script_path))[1]
25     println("Генерация из: $script_path")
26     # Чистый скрипт (без комментариев)
27     scripts_dir = scriptsdir(script_name)
28     Literate.script(script_path, scripts_dir; credit=false)
29     println(" / Чистый скрипт: $(scripts_dir)/$(script_name).jl")
30     # Quarto-документ
31     quarto_dir = projectdir("markdown", script_name);
32     Literate.markdown(script_path, quarto_dir;
33         flavor=Literate.QuartoFlavor(),
34         name=script_name, credit=false)
35     println(" / Quarto: $(quarto_dir)/$(script_name).qmd")
36     # Jupyter notebook
37     notebooks_dir = projectdir("notebooks", script_name)
38     Literate.notebook(script_path, notebooks_dir, name=script_name;
39         execute=false, credit=false)
40     println(" / Notebook: $(notebooks_dir)/$(script_name).ipynb")
41     println("\nГотово! Все файлы созданы.")
42 end
43
44 # Заныск
45 if abspath(PROGRAM_FILE) == @__FILE__
46     main()
47 end

```

Рисунок 4.6: «Скрипт scripts/tangle.jl»

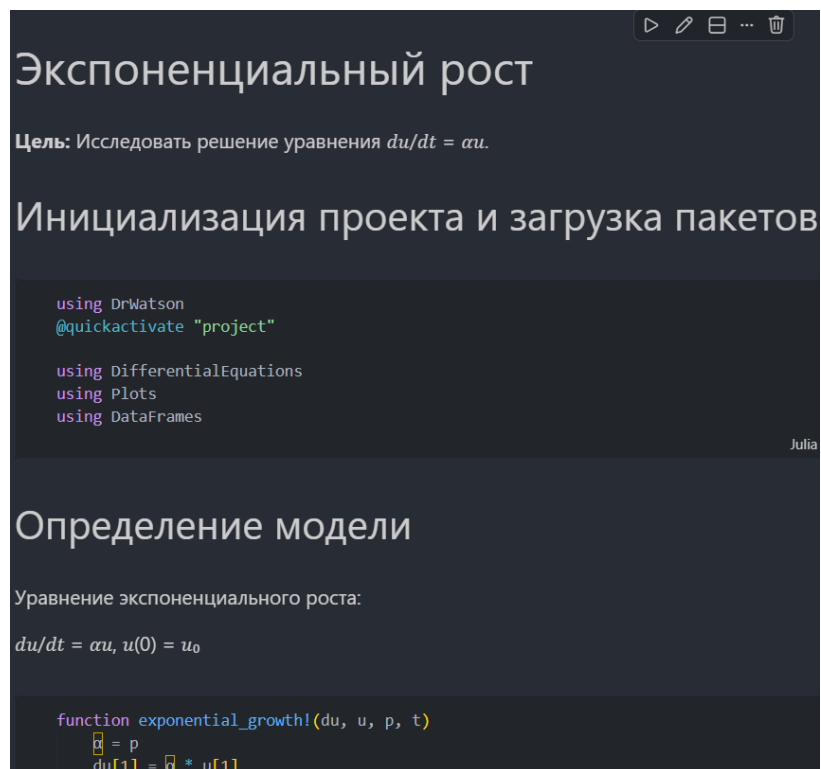


Рисунок 4.7: Формат `ipynb`

```

# Экспоненциальный рост
**Цель:** Исследовать решение уравнения  $du/dt = \alpha u$ .

## Инициализация проекта и загрузка пакетов

▷Run Cell | Run Next Cell
```{julia}
using DrWatson
@quickactivate "project"

using DifferentialEquations
using Plots
using DataFrames
```

## Определение модели
Уравнение экспоненциального роста:
 $du/dt = \alpha u, u(0) = u_0$ 

▷Run Cell | Run Next Cell | Run Above
```{julia}
function exponential_growth!(du, u, p, t)
 α = p
 du[1] = α * u[1]
end
```

## Первый запуск с параметрами по умолчанию
Зададим начальные параметры:

▷Run Cell | Run Next Cell | Run Above
```{julia}
u0 = [1.0] # начальная популяция
α = 0.3 # скорость роста

```

Рисунок 4.8: Формат qmd

Исследование не ограничивается одним значением параметров. Изменим программу так, чтобы она принимала набор параметров(рис. 4.9, рис. 4.10).

```

Admin@DESKTOP-KPF3U04 MINGW64 ~/work/study/2026-1/2026-1==study--security-modeling/2026-1--study--se
curity-modeling/labs/lab01/project (master)
$ julia --project=. scripts/02_exponential_growth.jl
Базовые параметры эксперимента:
α= 0.3
u0= [1.0]
saveat= 0.1
solver= Tsit5{typeof(OrdinaryDiffEqCore.trivial_limiter!), typeof(OrdinaryDiffEqCore.trivial_limiter!), FastBroadcast.Serial(), FastBroadcast.Serial(), OrdinaryDiffEqCore.trivial_limiter!}
experiment_name= base_experiment
tspan= (0.0, 10.0)
[Info: File C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\2026-1--study--security-modeling\labs\lab01\project\data\02_exponential_growth\single\exp_growth_experiment_name=base_experiment_saveat=0.1_α=0.3.jld2 does not exist. Producing it now...
[Info: File C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\2026-1--study--security-modeling\labs\lab01\project\data\02_exponential_growth\single\exp_growth_experiment_name=base_experiment_saveat=0.1_α=0.3.jld2 saved.

Результаты базового эксперимента:
Финальная популяция: 20.085448516186737
Время удвоения: 2.31
Файл результатов: C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\2026-1--study--security-modeling\labs\lab01\project\data\02_exponential_growth\single\exp_growth_experiment_name=base_experiment_saveat=0.1_α=0.3.jld2

=====
ПАРАМЕТРИЧЕСКОЕ СКАНИРОВАНИЕ
Всего комбинаций параметров: 5
Исследуемые значения α: [0.1, 0.3, 0.5, 0.8, 1.0]
=====
[Warning: Assignment to 'data' in soft scope is ambiguous because a global variable by the same name exists: 'data' will be treated as a new local. Disambiguate by using 'local data' to suppress this warning or 'global data' to assign to the existing global variable.
] @ C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\2026-1--study--security-modeling\labs\lab01\project\scripts\02_exponential_growth.jl:121
[Warning: Assignment to 'path' in soft scope is ambiguous because a global variable by the same name exists: 'path' will be treated as a new local. Disambiguate by using 'local path' to suppress this warning or 'global path' to assign to the existing global variable.
] @ C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\2026-1--study--security-modeling\labs\lab01\project\scripts\02_exponential_growth.jl:121
Прогресс: 1/5 | α = 0.1
Прогресс: 2/5 | α = 0.3
Прогресс: 3/5 | α = 0.5
Прогресс: 4/5 | α = 0.8
Прогресс: 5/5 | α = 1.0

Сводная таблица результатов:

```

Рисунок 4.9: «Скрипт scripts/02\_exponential\_growth.jl»

```

Admin@DESKTOP-KPF3U04 MINGW64 ~/work/study/2026-1/2026-1==study--security-modeling/2026-1--study--se
curity-modeling/labs/lab01/project (master)
$ julia --project=. scripts/tangle.jl scripts/02_exponential_growth.jl
Генерация из: scripts/02_exponential_growth.jl
[Info: generating plain script file from 'C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\2026-1--study--security-modeling\labs\lab01\project\scripts\02_exponential_growth.jl'
[Info: writing result to 'C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\2026-1--study--security-modeling\labs\lab01\project\scripts\02_exponential_growth\02_exponential_growth.jl'
✓ Чистый скрипт: C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\2026-1--study--security-modeling\labs\lab01\project\scripts\02_exponential_growth\02_exponential_growth.jl
[Info: generating markdown page from 'C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\2026-1--study--security-modeling\labs\lab01\project\scripts\02_exponential_growth.jl'
[Info: writing result to 'C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\2026-1--study--security-modeling\labs\lab01\project\markdown\02_exponential_growth\02_exponential_growth.qmd'
✓ Quarto: C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\2026-1--study--security-modeling\labs\lab01\project\markdown\02_exponential_growth\02_exponential_growth.qmd
[Info: generating notebook from 'C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\2026-1--study--security-modeling\labs\lab01\project\scripts\02_exponential_growth.jl'
[Info: writing result to 'C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\2026-1--study--security-modeling\labs\lab01\project\notebooks\02_exponential_growth\02_exponential_growth.ipynb'
✓ Notebook: C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\2026-1--study--security-modeling\labs\lab01\project\notebooks\02_exponential_growth\02_exponential_growth.ipynb

Готово! Все файлы созданы.

```

Рисунок 4.10: «Литературная реализация модели»

## 5 Экспоненциальный рост

**Цель:** Исследовать решение уравнения  $\frac{dx}{dt} = \lambda x$ .

### 5.1 Инициализация проекта и загрузка пакетов

---

```
using DrWatson
@quickactivate "project"

using DifferentialEquations
using Plots
using DataFrames
```

---

```
└ Warning: DrWatson could not find find a project file by recursively cl
x (given dir: C:\Users\Admin)
└ @ DrWatson C:\Users\Admin\.julia\packages\DrWatson\2QF5p\src\proje
```

### 5.2 Определение модели

Уравнение экспоненциального роста:  $\frac{dx}{dt} = \lambda x$ ,  $x(0) = x_0$

---

```
function exponential_growth!(du, u, p, t)
 du = p
```

```
du[1] = λ * u[1]
```

```
end
```

---

```
exponential_growth! (generic function with 1 method)
```

## 5.3 Первый запуск с параметрами по умолчанию

Зададим начальные параметры:

---

```
u0 = [1.0] #  
λ = 0.3 #  
tspan = (0.0, 10.0) #  
prob = ODEProblem(exponential_growth!, u0, tspan, λ)
sol = solve(prob, Tsit5(), saveat=0.1)
```

---

```
retcode: Success
```

```
Interpolation: 1st order linear
```

```
t: 101-element Vector{Float64}:
```

```
0.0
```

```
0.1
```

```
0.2
```

```
0.3
```

```
0.4
```

```
0.5
```

```
0.6
```

```
0.7
```

```
0.8
```

```
0.9
```

```
λ
```

9.2

9.3

9.4

9.5

9.6

9.7

9.8

9.9

10.0

u: 101-element Vector{Vector{Float64}}:

[1.0]

[1.030454533950446]

[1.0618365551529674]

[1.09417430287941]

[1.1274968605386673]

[1.161834232745065]

[1.197217347699063]

[1.2336780571872568]

[1.2712491879500347]

[1.3099646073864084]

⊠

[15.799688704153079]

[16.280848966976613]

[16.776672166300525]

[17.287605875776944]

[17.814109831385306]

[18.356655931432517]

[18.91572823655281]

[19.491822969707833]

[20.085448516186737]

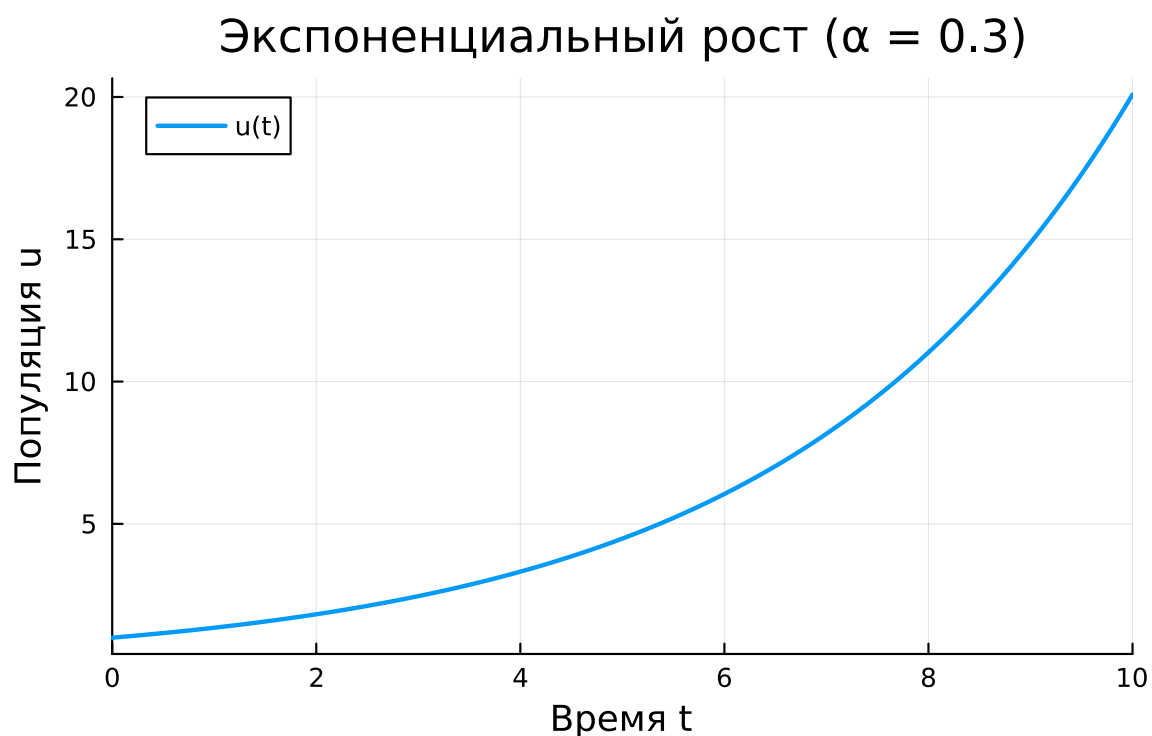
## 5.4 Визуализация результатов

Построим график решения:

---

```
plot(sol, label="u(t)", xlabel="Время t", ylabel="Популяция u",
 ↪ "u",
 title="Экспоненциальный рост (α = 0.3)", lw=2, legend=:topleft)
```

---



Сохраним график в папку plots

---

```
savefig(plotsdir("exponential_growth_α=0.3.png"))
```

---

```
"C:\\Users\\Admin\\work\\study\\2026-1\\2026-1==study--
security-modeling\\2026-1--study--security-modeling\\labs\\lab01\\p
```



## 5.5 Анализ результатов

Создадим таблицу с данными:

---

```
df = DataFrame(t=sol.t, u=first.(sol.u))
println("Популяция 5 клеток в таблице:")
println(first(df, 5))
```

---

Популяция 5 клеток в таблице:

5×2 DataFrame

Row	t	u
	Float64	Float64
1	0.0	1.0
2	0.1	1.03045
3	0.2	1.06184
4	0.3	1.09417
5	0.4	1.1275

Вычислим удвоение популяции:

---

```
u_final = last(sol.u)[1]
doubling_time = log(2) /
```

---

2.3104906018664844

## 5.6 Сохранение всех результатов

---

```
println("\nПопуляция клеток: ",
↵ round(doubling_time; digits=2))
```

---

XXXXXXXXXXXXXXXXXXXX XXXX XXXXXXXX: 2.31

## 6 Параметрическое исследование экспоненциального роста

### 6.1 Активация проекта и загрузка пакетов

ИЗМЕНЕНИЕ: Добавлен DrWatson для управления проектом и параметрами

---

```
using DrWatson
@quickactivate "project" # XXXXXXXX XXXXXXXX DrWatson
using DifferentialEquations
using DataFrames
using Plots
using JLD2
using BenchmarkTools
```

---

```
└ Warning: DrWatson could not find find a project file by recursively cl
 ✖ (given dir: C:\Users\Admin)
└ @ DrWatson C:\Users\Admin\.julia\packages\DrWatson\2QF5p\src\proje
```

Установка каталогов

---

```
script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
```

```
mkpath(datadir(script_name))
```

---

```
"C:\\Users\\Admin\\work\\study\\2026-1\\2026-1==study--
security-modeling\\2026-1--study--security-modeling\\labs\\lab01\\p
```

## 6.2 Определение модели

Модель:  $\frac{du}{dt} = \lambda \cdot u$

---

```
function exponential_growth!(du, u, p, t)
 λ = p.λ # * * * * *
 ↪ * * * * *
 du[1] = λ * u[1]
end
```

---

exponential\_growth! (generic function with 1 method)

## 6.3 Определение параметров в Dict

**ОСНОВНОЕ ИЗМЕНЕНИЕ:** Все параметры собраны в Dict для систематизации  
Базовый набор параметров (один эксперимент)

---

```
base_params = Dict(
 :u0 => [1.0], # * * * * *
 :λ => 0.3, # * * * * *
 :tspan => (0.0, 10.0), # * * * * *
 :solver => Tsit5(), # * * * * *
 :saveat => 0.1, # * * * * *
```

```

 :experiment_name => "base_experiment"
)

```

```

println("XXXXXXXX XXXXXXXXXXX XXXXXXXXXXXX:")
for (key, value) in base_params
 println(" $key= $value")
end

```

---

```

XXXXXXXX XXXXXXXXXXX XXXXXXXXXXXX:

```

```

 ρ= 0.3

```

```

 u0= [1.0]

```

```

 saveat= 0.1

```

```

 solver= OrdinaryDiffEqTsit5.Tsit5{typeof(OrdinaryDiffEqCore.trivial

```

```

 experiment_name= base_experiment

```

```

 tspan= (0.0, 10.0)

```

## 6.4 Функция-обертка для запуска одного эксперимента

**ИСПРАВЛЕНИЕ:** Возвращаем Dict со строковыми ключами

---

```

function run_single_experiment(params::Dict)
 @unpack u0, ρ, tspan, solver, saveat = params
 prob = ODEProblem(exponential_growth!, u0, tspan,
 ↪ (ρ=ρ,)) # XXXXXXX ρ XXXXXXX XXXXXXX
 sol = solve(prob, solver; saveat=saveat)
 final_population = last(sol.u)[1] # XXXXXXX XXXXXXXXXXXXXXX
 doubling_time = log(2) / ρ

```



XXXXXXXXXX XXXXXXXX XXXXXXXXXXXXX:

XXXXXXXXXX XXXXXXXXXXXX: 20.085448516186737

XXXXXX XXXXXXXX: 2.31

XXXX XXXXXXXXXXXX: C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\2026-1--study--security-modeling\labs\lab01\project\data\startup\single\exp\_growth\_experimen

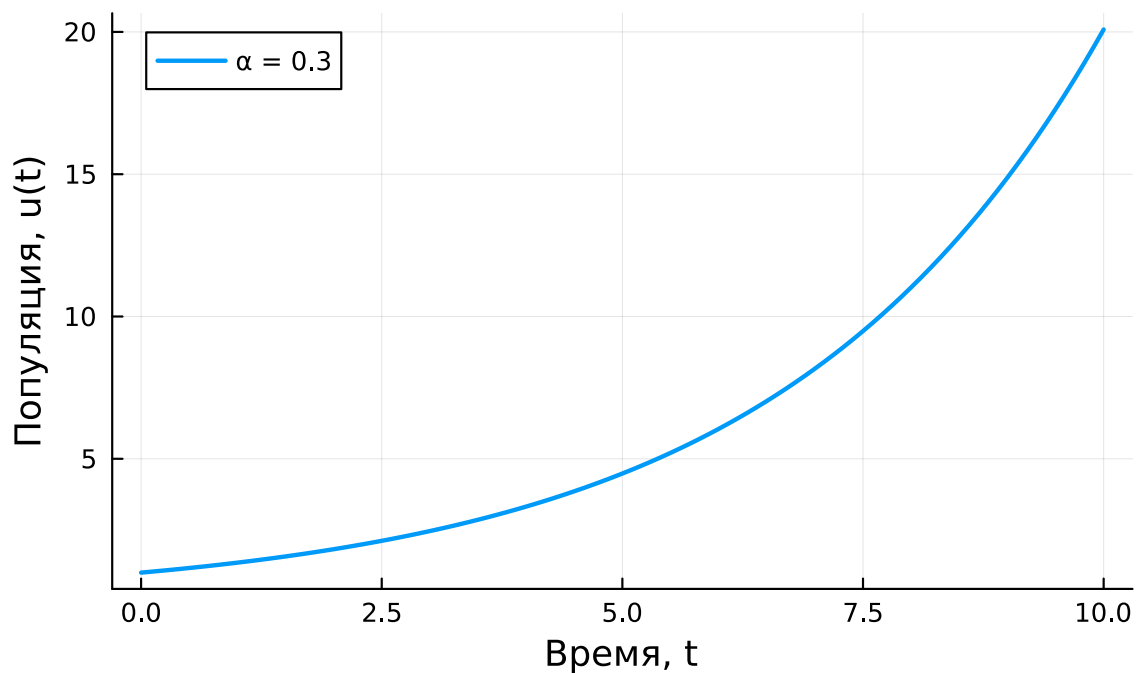
## 6.6 Визуализация базового эксперимента

---

```
p1 = plot(data["time_points"], data["population_values"],
 label=" $\dot{u} = \$(base_params[:u])$ ",
 xlabel="XXXXXX, t",
 ylabel="XXXXXXXXXX, u(t)",
 title="XXXXXXXXXXXXXXXXXXXX XXXX (XXXXXXXX XXXXXXXXXXXX)",
 lw=2,
 legend=:topleft,
 grid=true
)
```

---

## Экспоненциальный рост (базовый эксперимент)



Сохраним график в папку plots

---

```
savefig(plotsdir(script_name, "single_experiment.png"))
```

---

```
"C:\\Users\\Admin\\work\\study\\2026-1\\2026-1==study--
security-modeling\\2026-1--study--security-modeling\\labs\\lab01\\p
```

## 6.7 Параметрическое сканирование

**НОВАЯ СЕКЦИЯ:** Исследование влияния параметра  $\alpha$

Сетка параметров для сканирования

---

```
param_grid = Dict(
 :u0 => [[1.0]], #
 :α => [0.1, 0.3, 0.5, 0.8, 1.0], #
 ↪
)
```



```

:tspan => [(0.0, 10.0)], # XXXXXXXXXX XXXXXXXX XXXXXXXX
:solver => [Tsit5()], # XXXXXXXXXX XXXXXX XXXXXXXX
:saveat => [0.1], # XXXXXXXXXX XXX XXXXXXXXX
:experiment_name => ["parametric_scan"]
)

```

---

Dict{Symbol, Vector} with 6 entries:

```

:␣ => [0.1, 0.3, 0.5, 0.8, 1.0]
:u0 => [[1.0]]
:saveat => [0.1]
:solver => [Tsit5{typeof(trivial_limiter!), typeof(trivial_li
:experiment_name => ["parametric_scan"]
:tspan => [(0.0, 10.0)]

```

Генерация всех комбинаций параметров

---

```

all_params = dict_list(param_grid)
println("\n" * "="^60)
println(XX)
println(XXXXXX XXXXXXXXXX XXXXXXXXXX: ", length(all_params))
println(XXXXXXXXXXXXXXXX XXXXXXXXX ␣: ", param_grid[:␣])
println("="^60)

```

---

```

=====
XX
XXXXXX XXXXXXXXXX XXXXXXXXXX: 5
XXXXXXXXXXXXXXXX XXXXXXXXX ␣: [0.1, 0.3, 0.5, 0.8, 1.0]
=====

```

## 6.8 Запуск всех экспериментов и сбор результатов

НОВАЯ СЕКЦИЯ: Автоматический запуск и сохранение всех вариантов

---

```
all_results = []
all_dfs = []
for (i, params) in enumerate(all_params)
println("#####: $i/$(length(all_params)) | =
↳ $(params[:])")
data, path = produce_or_load(
 datadir(script_name, "parametric_scan"), #
 params, #
 run_single_experiment, #
 prefix = "scan", #
 tag = false,
 verbose = false #
 ↳
) #
result_summary = merge(
 params,
 Dict(
 :final_population => data["final_population"],
 :doubling_time => data["doubling_time"],
 :filepath => path #
)
) #
↳ , data -)

push!(all_results, result_summary)
```

```

df = DataFrame(
 t = data["time_points"],
 u = data["population_values"],
 ⬜ = fill(params[:⬜], length(data["time_points"]))
) # ██████████ ███████ ████████ ███ ████████████████
push!(all_dfs, df)
end

```

---

```

██████████: 1/5 | ⬜ = 0.1
██████████: 2/5 | ⬜ = 0.3
██████████: 3/5 | ⬜ = 0.5
██████████: 4/5 | ⬜ = 0.8
██████████: 5/5 | ⬜ = 1.0

```

## 6.9 Анализ и визуализация результатов сканирования

**НОВАЯ СЕКЦИЯ:** Сравнительный анализ всех экспериментов Сводная таблица результатов

```

results_df = DataFrame(all_results)
println("\n██████████ ███████ ████████████████:")
println(results_df[!, [:⬜, :final_population,
 ↵ :doubling_time]])

```

---

```

██████████ ████████████████████:

```

5×3 DataFrame

Row		final_population	doubling_time
	Float64	Float64	Float64
1	0.1	2.71828	6.93147
2	0.3	20.0854	2.31049
3	0.5	148.409	1.38629
4	0.8	2980.57	0.866434
5	1.0	22021.0	0.693147

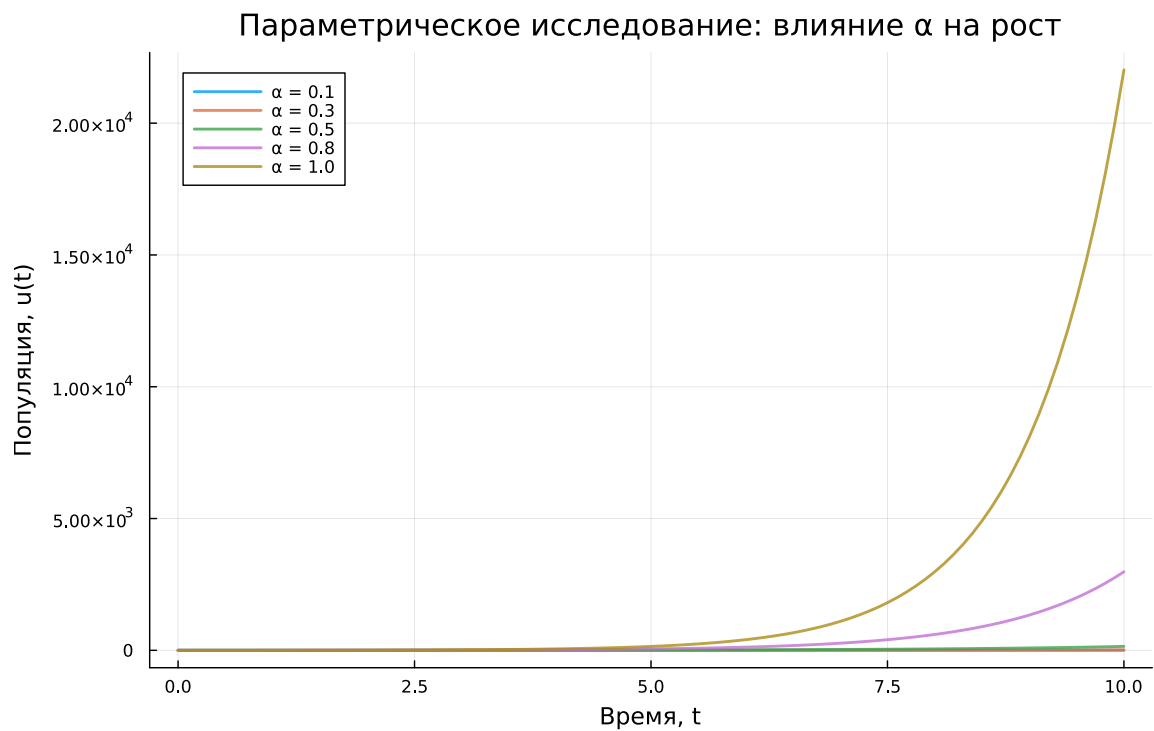
Сравнительный график всех траекторий

```
p2 = plot(size=(800, 500), dpi=150)
for params in all_params
 data, _ = produce_or_load(
 datadir(script_name, "parametric_scan"),
 params,
 run_single_experiment,
 prefix = "scan"
) # XXXXXXXXXXXXXXXXXXXX (XXXX XXX XXXXX XX XXXXXX)
 plot!(p2, data["time_points"],
 ↪ data["population_values"],
 label="X = $(params[:X])",
 lw=2,
 alpha=0.8
)
end
plot!(p2,
 xlabel="XXXXXXXX, t",
```

```

ylabel="Популяция, u(t)",
title="Параметрическое исследование: влияние α на рост",
legend=:topleft,
grid=true
)

```



Сохраним график в папку plots

```

savefig(plotsdir(script_name,
 ↪ "parametric_scan_comparison.png"))

```

"C:\\Users\\Admin\\work\\study\\2026-1\\2026-1==study--  
security-modeling\\2026-1--study--security-modeling\\labs\\lab01\\p

График зависимости времени удвоения от α

```

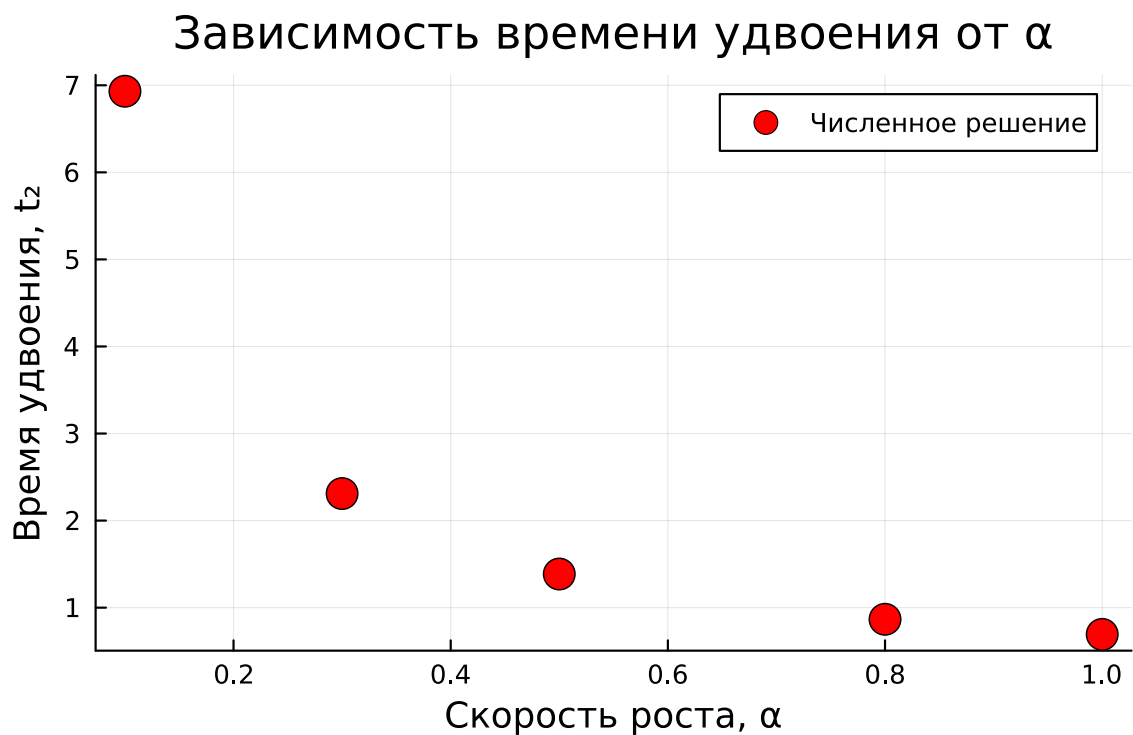
p3 = plot(results_df.⌈, results_df.doubling_time,

```

```

seriestype=:scatter,
label="Численное решение",
xlabel="Скорость роста, α",
ylabel="Время удвоения, t₂",
title="Зависимость времени удвоения от α",
markersize=8,
markercolor=:red,
legend=:topright
)

```



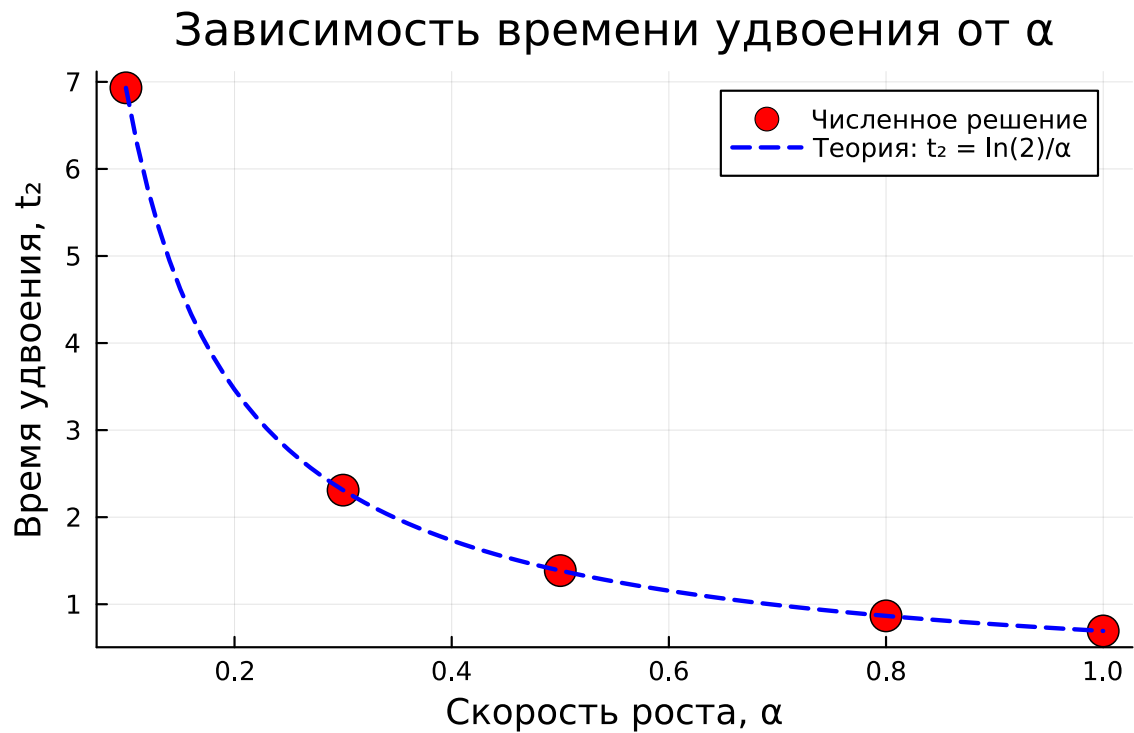
Теоретическая кривая:  $t_2 = \ln(2)/\alpha$

```

α_range = 0.1:0.01:1.0
plot!(p3, α_range, log(2) ./ α_range,
label="Теоретическая кривая: t₂ = ln(2)/α",
lw=2,

```

```
linestyle=:dash,
linecolor=:blue
)
```



Сохраним график в папку plots

```
savefig(plotsdir(script_name,
↪ "doubling_time_vs_alpha.png"))
```

```
"C:\\Users\\Admin\\work\\study\\2026-1\\2026-1==study--
security-modeling\\2026-1--study--security-modeling\\labs\\lab01\\p
```

## 6.10 Бенчмаркинг с разными параметрами

**ИЗМЕНЕНИЕ:** Бенчмаркинг для разных значений  $\alpha$

---

```

println("\n" * "="^60)
println("XXXXXXXXXX XX XXXXXX XXXXXXXX X")
println("="^60)
benchmark_results = []

for x_value in param_grid[:,x]
 bench_params = Dict(
 :u0 => [1.0],
 :x => x_value,
 :tspan => (0.0, 10.0),
 :solver => Tsit5(),
 :saveat => 0.1
) # XXXXXXXXXXXXXXXXXXXX XXX XXXXXXXXXXXX
 function benchmark_run() # XXXXXXXX XX XXXXXXXXXXXX
 prob = ODEProblem(exponential_growth!,
 bench_params[:u0],
 bench_params[:tspan],
 (x=bench_params[:x],))
 return solve(prob, bench_params[:solver];
 saveat=bench_params[:saveat])
 end
 println("\nXXXXXXXXXX XX X = $x_value:")
 b = @benchmark $benchmark_run() samples=100 evals=1 #
 ↪ XXXXXXXX XXXXXXXXXXXX
 push!(benchmark_results, (x=x_value,
 ↪ time=median(b).time/1e9)) # XXXXXX X XXXXXXXXXXXX
 println(" XXXXXXXX XXXXXX: ", round(median(b).time/1e9;
 digits=4), " XXXX")

```



end

---

```
=====
XXXXXXXXXXXXXXXX XXX XXXXXXXXXXX XXXXXXXXXXX X
=====
```

```
XXXXXXXXXXXX XXX X = 0.1:
XXXXXXXXXXXX XXXXXXX: 0.0 XXX
```

```
XXXXXXXXXXXX XXX X = 0.3:
XXXXXXXXXXXX XXXXXXX: 0.0 XXX
```

```
XXXXXXXXXXXX XXX X = 0.5:
XXXXXXXXXXXX XXXXXXX: 0.0 XXX
```

```
XXXXXXXXXXXX XXX X = 0.8:
XXXXXXXXXXXX XXXXXXX: 0.0 XXX
```

```
XXXXXXXXXXXX XXX X = 1.0:
XXXXXXXXXXXX XXXXXXX: 0.0 XXX
```

График зависимости времени вычисления от  $\alpha$

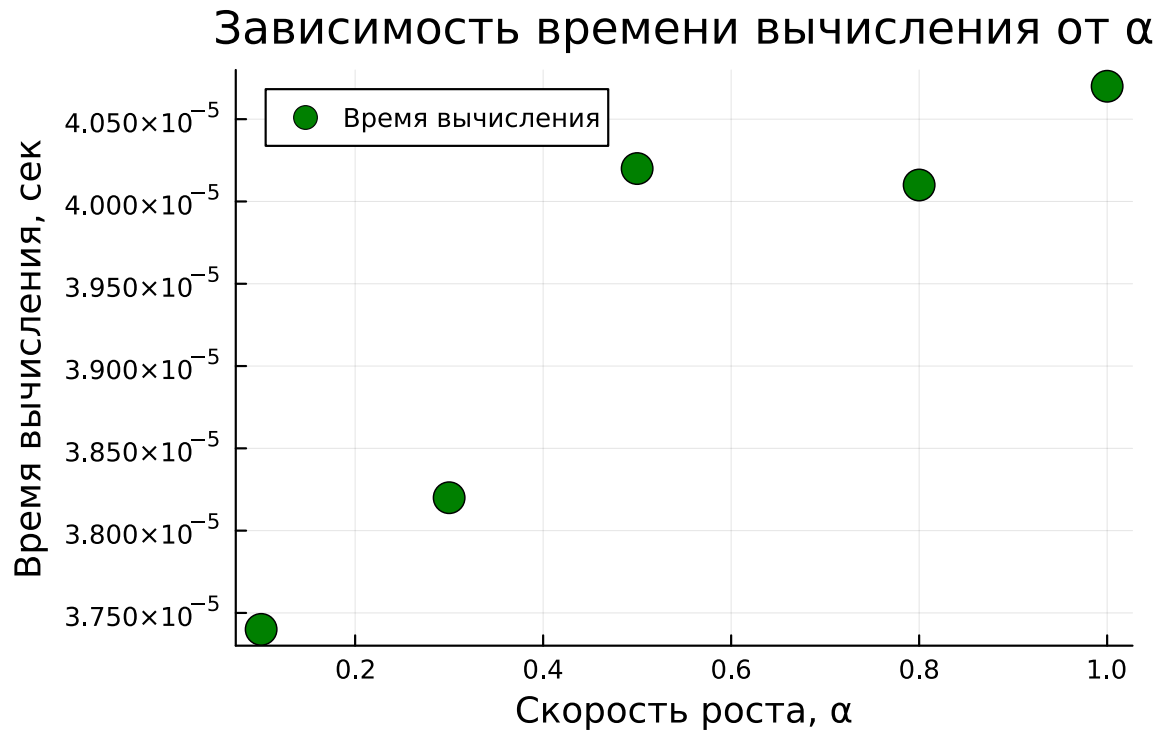
---

```
bench_df = DataFrame(benchmark_results)
p4 = plot(bench_df.X, bench_df.time,
 seriestype=:scatter,
 label="XXXXXXXX XXXXXXXXXXX",
 xlabel="XXXXXXXX XXXXXXX, X",
```

```

ylabel="Время вычисления, сек",
title="Зависимость времени вычисления от α ",
markersize=8,
markercolor=:green,
legend=:topleft
)

```



Сохраним график в папку plots

```

savefig(plotsdir(script_name,
↪ "computation_time_vs_alpha.png"))

```

```

"C:\\Users\\Admin\\work\\study\\2026-1\\2026-1==study--
security-modeling\\2026-1--study--security-modeling\\labs\\lab01\\p

```

## 6.11 Сохранение всех результатов

**НОВАЯ СЕКЦИЯ:** Сохранение сводных данных для последующего анализа

---

```
@save datadir(script_name, "all_results.jld2")
 ↪ base_params param_grid all_params results_df bench_df
@save datadir(script_name, "all_plots.jld2") p1 p2 p3 p4
println("\n" * "="^60)
println("XXXXXXXXXXXXXXXX XXXXXX XXXXXXXXXXXX")
println("="^60)
println("\nXXXXXXXXXXXXXXXX XXXXXXXXXXXX X:")
println(" • data/${script_name}/single/ - XXXXXXXX
 ↪ XXXXXXXXXXXXXXXX")
println(" • data/${script_name}/parametric_scan/ -
 ↪ XXXXXXXXXXXXXXXXXXXX XXXXXXXXXXXXXXXX")
println(" • data/${script_name}/all_results.jld2 - XXXXXXXX
 ↪ XXXXXXXX")
println(" • plots/${script_name}/ - XXX XXXXXXXX")
println(" • data/${script_name}/all_plots.jld2 - XXXXXXXX
 ↪ XXXXXXXXXXXXXXXX")
println("\nXXX XXXXXXX XXXXXXXXXXXXXXX XXXXXXXXXXXXXXX:")
println(" using JLD2, DataFrames")
println(" @load
 ↪ \"data/${script_name}/all_results.jld2\")
println(" println(results_df)")
```

---

=====

XXXXXXXXXXXXXXXX XXXXXX XXXXXXXXXXXX

=====

XXXXXXXXXX XXXXXXXX X:

- data/startup/single/ - XXXXXXXX XXXXXXXXX
- data/startup/parametric\_scan/ - XXXXXXXXXXXXXXXXXXXX XXXXXXXXXXXXXXXX
- data/startup/all\_results.jld2 - XXXXXXXX XXXXXXXX
- plots/startup/ - XXX XXXXXXXX
- data/startup/all\_plots.jld2 - XXXXXXXX XXXXXXXXX

XXX XXXXXXXX XXXXXXXXXXXXXXX XXXXXXXXXXXXXXX:

```
using JLD2, DataFrames
@load "data/startup/all_results.jld2"
println(results_df)
```

## 7 Выводы

В результате работы были освоены принципы literate-программирования и воспроизводимых вычислений, а также реализована автоматическая генерация различных форматов отчётов из единого исходного Julia-скрипта с использованием пакетов DrWatson и Literate.

# Список литературы

1. DrWatson.jl Documentation. — URL: <https://juliadynamics.github.io/DrWatson.jl/stable/> (дата обр. 20.08.2026).
2. JuliaLang. — URL: <https://julialang.org/> (дата обр. 20.08.2026).
3. Literate.jl Documentation. — URL: <https://fredrikekre.github.io/Literate.jl/v2/> (дата обр. 20.08.2026).